SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME:OPERATING SYSTEMS

COURSE CODE: CSA0407

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

QUESTIONS: 5 **Total Marks : 50**

Annexure A

Scenario based Questions

Code of Conduct:

I am Pugazharasan K (192511341) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Pugazh.K

Question 1: CPU Scheduling in a Real-Time Ticket Booking System (10 Marks)

1. Introduction:

CPU scheduling is the process of selecting a process from the ready queue for execution by the CPU. It helps improve CPU utilization, reduce waiting time, and increase system performance.

2. Suitable Scheduling Algorithm:

Priority Scheduling is suitable for the railway ticket booking system because the processes have different levels of importance.

- P1 – Payment gateway transactions: High priority
- P2 – Seat availability checks: Medium priority
- P3 – Ticket confirmation and printing: Low priority

3. Justification

1. High-priority payment transactions are executed first.
2. Medium-priority seat availability checks are executed next.
3. Low-priority ticket confirmation tasks are executed after higher-priority tasks.
4. Preemptive priority scheduling allows important processes to execute immediately when necessary.
5. Efficient scheduling reduces waiting time for important transactions and improves responsiveness.

However, ticket confirmation may experience starvation if high-priority processes continuously arrive. Aging can be used to prevent this problem.

4. Sample Execution Order

Assuming all three processes are ready at the same time:

Order	Process	Priority
1	P1 – Payment	High
2	P2 – Seat availability	Medium
3	P3 – Ticket confirmation	Low

Execution order: **P1 → P2 → P3**

The actual execution order may change if processes arrive at different times or preemption occurs.

5. Drawback

- The main drawback of priority scheduling is starvation. Low-priority processes may wait indefinitely when high-priority processes continuously arrive.

6. Conclusion

- Priority scheduling is suitable for a railway ticket booking system because it gives preference to important transactions. Using aging along with priority scheduling helps reduce starvation and improve system responsiveness.

Question 2: Deadlock in a Banking Transaction Server (10 Marks)

1. Introduction:

A deadlock occurs when two or more processes wait indefinitely for resources held by one another. It prevents the processes from completing their operations.

2. Resource Allocation Graph (RAG):

The given situation is:

- T1 holds resource A and requests B.
- T2 holds resource B and requests C.
- T3 holds resource C and requests A.

Text-based Resource Allocation Graph:

T1 → B → T2 → C → T3 → A → T1

Here, an arrow from a process to a resource represents a request, and an arrow from a resource to a process represents an allocation.

The complete relationships are:

A → T1 → B → T2 → C → T3 → A

3. Identification of Deadlock:

Yes, a deadlock has occurred, assuming each resource has only one instance and the locks cannot be forcibly taken away.

The circular wait is:

1. T1 holds A and waits for B.
2. T2 holds B and waits for C.
3. T3 holds C and waits for A.
4. T1 cannot proceed until T2 releases B.
5. T2 cannot proceed until T3 releases C.

6. T3 cannot proceed until T1 releases A.

Therefore, all three transactions wait indefinitely.

4. Recommended Technique: Deadlock Prevention:

1. Deadlock prevention eliminates at least one of the necessary conditions for deadlock.
2. A suitable method is to enforce a fixed resource-locking order.
3. For example, all transactions must acquire resources in the order:
4. A → B → C
5. If a transaction needs multiple resources, it must request them in this order and release them after completing its operation.
6. This prevents circular wait and avoids the deadlock.

5. Redesign of Locking Order:

- T1 must acquire A before B.
- T2 must acquire B only after following the established resource order.
- T3 must acquire resources according to the same order.
- Transactions must release their locks after completing the operation.
- If all required locks cannot be acquired safely, the transaction should wait without holding conflicting locks or retry later.

6. Advantages:

- Prevents circular wait.
- Improves transaction reliability.
- Reduces indefinite waiting.
- Maintains database consistency.
- Improves banking server performance.

7. Conclusion:

- The banking server experiences deadlock because T1, T2, and T3 form a circular wait. Enforcing a consistent resource-locking order prevents future deadlocks and improves transaction processing.

Question 3: Mobile OS App Switching Lag (10 Marks)

1. Introduction:

A mobile operating system manages multiple applications and allocates CPU time to them. Efficient scheduling is necessary for smooth app switching, audio playback, and better system performance.

The given system uses Round Robin scheduling with a time quantum of 20 ms and has 20 background applications active.

2. Why Does the CPU Remain Idle?

The CPU may remain idle despite multiple active applications because:

1. Many applications may be waiting for network or disk I/O.
2. Background applications may be blocked while waiting for events.
3. Applications may be waiting for locks or other resources.
4. The ready queue may temporarily become empty.
5. The operating system may be performing power-saving or idle-state management.

Important: The presence of many active applications does not guarantee that processes are always ready to execute.

3. Effect of the Time Quantum:

1. The current Round Robin time quantum is 20 ms.
2. A time quantum determines how long a process can execute before the scheduler switches to another ready process.
3. A 20 ms quantum may hurt responsiveness when interactive applications need quick CPU access. However, it reduces the frequency of time-slice-based context switches compared with a smaller quantum.
4. The time quantum alone does not explain CPU idle time; I/O waiting and the number of ready processes must also be examined.

4. Recommended Scheduling Method:

A suitable recommendation is **preemptive priority scheduling** for foreground and background applications.

- Give higher priority to the foreground application.
- Give suitable priority to the music player so that audio playback remains smooth.
- Assign lower priority to non-essential background applications.
- Use aging to prevent background applications from waiting indefinitely.

Alternatively, Round Robin can be retained with a smaller quantum, such as 5–10 ms, if testing shows that improved responsiveness outweighs the additional context-switch overhead.

5. How the Change Improves Performance:

1. Foreground applications receive CPU time more quickly.
2. Background audio playback receives appropriate scheduling priority.
3. App switching becomes more responsive.
4. Waiting time for interactive applications decreases.
5. Efficient scheduling can improve throughput and the user experience.

6. Conclusion:

- The mobile OS may experience idle time because applications are waiting for I/O or other events. Preemptive priority scheduling, suitable time-quantum tuning, and background task management can reduce switching lag and improve system responsiveness.

Question 4: Deadlock Risk in an Operating System Update (10 Marks)

1. Introduction:

During an operating system update, several modules execute simultaneously and share resources. If these modules hold resources while waiting for one another, a deadlock may occur.

2. Given Information:

- M1 copies update files and holds R1 while waiting for R2.
- M2 installs drivers and holds R2 while waiting for R3.
- M3 updates the registry and holds R3 while waiting for R1.

Resources:

- R1 – Disk I/O
- R2 – Kernel lock
- R3 – Configuration lock

3. Can Deadlock Occur?

Yes. Assuming each resource has one instance and cannot be forcibly taken away, the sequence can lead to deadlock.

4. Circular Wait Condition:

The resource relationships are:

M1 → R2 → M2 → R3 → M3 → R1 → M1

The complete cycle is:

R1 → M1 → R2 → M2 → R3 → M3 → R1

Explanation:

1. M1 holds R1 and waits for R2.

2. M2 holds R2 and waits for R3.
3. M3 holds R3 and waits for R1.
4. M1 cannot proceed until M2 releases R2.
5. M2 cannot proceed until M3 releases R3.
6. M3 cannot proceed until M1 releases R1.

Therefore, all three modules are blocked, creating a deadlock.

5. Banker's Algorithm for Deadlock Avoidance:

Banker's algorithm avoids unsafe resource allocations by checking whether the system can complete all processes in a safe order.

The steps are:

1. Identify the available resources.
2. Record the maximum resource requirements of each module.
3. Calculate the remaining need of each module.
4. Check whether the available resources satisfy the needs of any unfinished module.
5. If a module can finish, assume it completes and releases its resources.
6. Repeat the process to determine whether all modules can finish.
7. Grant a resource request only if the resulting state remains safe.

If a request would produce an unsafe state, the operating system delays the request until resources become available.

6. Prioritizing Modules:

The operating system can improve update performance by:

- Prioritizing essential update tasks.
- Executing dependent modules in the correct order.
- Avoiding unnecessary parallel resource requests.

- Giving CPU time to modules that are ready to execute.
- Preventing low-priority background tasks from delaying critical update operations.

Resource allocation must still follow a safe locking policy to prevent deadlocks.

7. Conclusion:

- The update modules may enter deadlock because of circular waiting for disk I/O, kernel locks, and configuration locks. Banker's algorithm can avoid unsafe allocations when maximum resource requirements are known, while appropriate module scheduling improves update performance.

Question 5: Cloud Server CPU Utilization Analysis (10 Marks)

1. Introduction:

CPU utilization measures the percentage of time the CPU is busy executing processes. In cloud computing, efficient CPU utilization improves throughput and makes better use of virtual machines and server resources.

2. Given Information:

- Probability of I/O waiting, ($p = 0.8$)
- Number of virtual machines, ($n = 7$)
- Each VM's processes spend approximately 80% of their time waiting for network or disk operations.

3. Calculate CPU Utilization:

The standard formula used for this type of problem is:

$$
U = 1-p^n
$$

Where:

- (U) = CPU utilization
- (p) = probability that a process is waiting for I/O
- (n) = number of independent processes or VMs

Substituting the values:

$$
U = 1-(0.8)^7
$$

$$
U = 1-0.2097152
$$

$$
U = 0.7902848
$$

Converting into percentage:

$$
U = 0.7902848 \times 100
$$

CPU Utilization = 79.03% (approximately)

4. Interpretation:

The calculated CPU utilization is approximately 79.03%.

This indicates that the CPU is busy for a substantial portion of the time, while some capacity remains available.

The result is based on the standard model that assumes independent I/O waiting behavior and that at least one of the seven VMs can provide runnable work. Actual utilization may differ due to workload, scheduling, and resource constraints.

5. Effect of Increasing or Decreasing VMs

Increasing the number of VMs:

- May increase CPU utilization when the server has spare capacity.
- Can improve throughput by allowing another VM to use the CPU while other VMs wait for I/O.
- May cause resource contention if too many VMs are added.
- Can reduce performance if CPU and memory resources become overloaded.

Decreasing the number of VMs:

- May reduce resource contention.
- Can improve performance when the server is overloaded.
- May reduce overall throughput if too few VMs remain to use the available CPU.

Therefore, the number of VMs should be selected according to workload and available server resources.

6. Two OS-Level Strategies:

Strategy 1: Improve CPU Scheduling

- Use an appropriate scheduling algorithm.
- Schedule runnable VMs efficiently.
- Avoid unnecessary scheduling overhead.
- Balance CPU time fairly among VMs.

Strategy 2: Improve I/O Performance

- Use asynchronous I/O where appropriate.
- Optimize disk and network operations.
- Use buffering and batching to reduce unnecessary I/O overhead.
- Reduce unnecessary blocking operations.

These strategies help keep the CPU busy when runnable work is available and improve overall throughput.

7. Conclusion:

- Using the formula (U=1-p^n), the estimated CPU utilization is 79.03%.

  Increasing the number of VMs can improve throughput when spare capacity exists, but excessive VM allocation can cause contention. Efficient CPU scheduling and I/O optimization can further improve cloud server performance.

RUBRICS FOR CALCULATION- Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

TOTAL MARKS AWARDED **FACULTY SIGNATURE**